Singly Linked List – Delete From Beginning –Space Complexity and Algorithm.

Program:

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
```

```
    }
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    newNode->next = nullptr;
    if (*head == nullptr)
    {
        *head = newNode;
    }
    else
    {
        Node *current = *head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
}

void insertAtPosition(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == nullptr)
    {
```

```
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    if (position == 1)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}

void insertAtPosition2(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (position == 0)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 0)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
```

```
        int i = 0;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }

        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}

void deleteFromBeginning(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *temp = *head;
        *head = (*head)->next;
        cout << "Deleted node: " << temp->data << endl;
        temp->next = nullptr;
        free(temp);
        temp = nullptr;
    }
    cout << "Node is deleted successfully." << endl;
}
```

```
int main()
{
   createEmptyList(&head);
    while (1)
    {

        cout << "1. Insert at the beginning" << endl;
        cout << "2. Insert at the end" << endl;
        cout << "3. Insert at a position,indexing node from 1 to N" << endl;
        cout << "4. Insert at a position,indexing node from 0 to N-1" << endl;
        cout << "5. Delete from the beginning" << endl;

        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
      case 2:
      {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtEnd(&head, data);
            break;
      }
     case 3:
        {
```

```
            int data,position;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition(&head, data, position);
            break;
           }

        case 4:
        {
            cout << "Insert at a position,indexing node from 0 to N-1:" << endl;
            int data, position;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition2(&head, data, position);

            break;
        }
        case 5:
        {
            deleteFromBeginning(&head);
            break;
        }
       default:
        {
            cout << "Invalid choice." << endl;
            break;
        }
        }
    }
  return 0;
}
```

Space Complexity Analysis

1. ***Input Space***:

A. Parameter – Only Definition

The input to the function is a double pointer Node ** head, which points to the head of the linked list. This requires constant space, i.e., O(1).

• Single parameter head (type Node **) → one machine word.

• Complexity: Θ(1).

This view considers only the space taken by the function's parameters. The single parameter `Node **head` is a pointer-to-a-pointer, which has a fixed, constant size. Therefore, the input space complexity is **O(1)**.

B. Full – Data (inclusive) Definition

This more common definition includes the entire data structure accessible from the parameters. If the linked list has **`n` nodes** before the deletion, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

• Same single word for head (Θ(1)).

• Plus every node reachable from *head.

• If the list currently contains n nodes, the input occupies Θ(n) bytes.

• Complexity: Θ(n).

2. ***Auxiliary Space***:

- ***Auxiliary space refers to any extra space or temporary space used by the algorithm. In this function, the auxiliary space is primarily used for the temporary pointer***

$Node * temp$.

- $The\ temp\ pointer\ is\ used\ to\ temporarily\ store\ the\ address\ of\ the\ node\ to\ be\ deleted.$ $This\ also\ requires\ constant\ space, i.e., O(1)$.

Auxiliary space is the extra memory an algorithm uses during its execution, not counting the input.

1. **`Node *temp`**: The function declares a single temporary pointer, `temp`, to hold the address of the node being deleted. This pointer occupies a constant amount of space.
2. **`free(temp)`**: This is a memory deallocation function. It *releases* memory back to the system; it does not consume extra working space for the algorithm.

Since the function only needs one extra pointer to do its job, regardless of how long the linked list is, the amount of extra memory used is constant.

Therefore, the auxiliary space complexity is **Θ(1)**.

• Local pointer temp and a few temporary scalars produced by the compiler → Θ(1).

• No additional dynamic allocations are performed (the function actually frees memory).

• No recursion, no auxiliary data structures.

• Complexity: Θ(1).

Algorithm

$DELBEG(START)$

1. $[LIST\ is\ Empty?]$

$If\ START = NULL, then:$

$Write:$ ``Error: List is empty.``

$Return\ and\ EXIT.$

$[DELETE\ NODE]$

$ELSE$

$Set\ TEMP := START\ [TEMP\ is\ a\ temporary\ pointer\ to\ store\ the\ address\ of\ the\ node\ to\ be\ deleted.]$

$Set\ START := LINK[TEMP]$ [$Update\ START\ to\ point\ to\ the\ next\ node\ in\ the\ list.$]

$Write$: "$Deleted\ node$: " $followed\ by\ INFO[TEMP]$ [$Print\ the\ data\ of\ the\ node\ being\ deleted.$]

$Set\ LINK[TEMP] := NULL$ [$Disconnect\ the\ node\ from\ the\ list.$]

$FREE(TEMP)$ [$Deallocate\ the\ memory\ used\ by\ the\ node.$]

$TEMP := NULL$[$Sets\ Dangling\ pointer\ TEMP\ to\ NULL$]

[$End\ of\ If\ structure$]

2. [$Write\ Statement\ on\ Confirmation\ of\ Deletion$]$Write$: "$Node\ is\ deleted\ successfully.$"

[$End\ of\ Algorithm.$]